
Coded Elastic Computing

Yaoqing Yang, Pulkit Grover, Soumya Kar
 Carnegie Mellon University
 {yyaoqing, pgrover, soumyak}@andrew.cmu.edu

Matteo Interlandi, Saeed Amizadeh, Markus Weimer
 Microsoft
 {mainterl, saamizad, mweimer}@microsoft.com *

Abstract

Cloud providers have recently introduced low-priority machines to reduce the cost of computations. Exploiting such opportunity for machine learning tasks is challenging inasmuch as low-priority machines can *elastically* leave (through *preemption*) and join the computation at any time. In this paper, we design a new technique called *coded elastic computing* enabling distributed machine learning computations over elastic resources. The proposed technique allows machines to transparently leave the computation without sacrificing the algorithm-level performance, and, at the same time, flexibly reduce the workload at existing machines when new machines join the computation. Thanks to the redundancy provided by encoding, our approach is able to achieve similar computational cost as the original (uncoded) method when all machines are present; the cost gracefully increases when machines are preempted and reduces when machines join. We test the performance of the proposed technique on two minibenchmark experiments, namely elastic matrix multiplications and linear regression. Our preliminary results show improvements over several existing techniques.

1 Introduction

New offerings from cloud-service providers allow exploiting under-utilized Virtual Machines (VMs) at a fraction of the original cost [1, 3]. For example, Azure Batch provides low-priority machines at about one-fifth of the cost of ordinary virtual machines [4]. Similarly, Amazon Spot Instances provide machines at market price with a discount which can reach up to 90% with respect to regular on-demand prices [2]. Such offerings, however, have the drawback that machines can be preempted at any time if a high-priority job appears. This, in turn, will surface as a computation failure at the application level. While common distributed machine learning frameworks are already built with fault-tolerance [5, 48], they often assume that failures are transient and rare. Due to this assumption, machine failures are often recovered by a “stop-the-world” scheme whereby the entire system is forced to wait until regular execution on the failure machines are restored from previous state (eventually on new machines). The above assumptions, however, do not necessarily hold for failures due to machines being preempted because (1) these failures are permanent and local data may not be accessible anymore; (2) several machines can be preempted altogether (up to 90% [45]); (3) these failures add up to transient failures, therefore leading to more frequent disruptions during computation; and (4) the computational framework may need to acquire additional machines to compensate, meaning that data has to be copied on the new machines which will likely become stragglers for the running computation. In practice, we observed situations at scale where the stop-the-

*Some preliminary results of the paper have been presented at the Workshop on Systems for ML and Open Source Software at NeurIPS 2018.

world scheme results in zero computation progress because, by the time a failure is recovered, a new failure occurs. This results in the necessity to build an elastic run-time framework [17, 49] and related failure-aware algorithms which can continue the computation and flexibly adapt in the presence of failures. Another possible technique to deal with preemption is to view the preempted machines as erasure-type faults and ignore them. Although machine learning algorithms are robust to small transient faults, simply ignoring the computational results in these permanently-failed preempted machines may result in algorithmic-level performance loss [49]. The influence of ignoring the computation results for the preemption type of faults is also more severe than usual computation faults because the number of failures can be really large [45]. Similarly, even if the data are redundant in some applications, ignoring partial results may still lead to reduced confidence levels on the prediction accuracy. It may also not be desirable from a customer’s perspective who often requires the full dataset to be present during the entire training process in order to achieve the highest accuracy.

In order to deal with the aforementioned problems, in this paper we propose *coded elastic computing*: a novel distributed learning framework allowing users to train their machine learning models over preemptable machines. In our coded elastic computing framework machines are allowed to arbitrarily join or leave during a distributed iterative learning task thanks to the introduction of redundancy in the computation. Although coded elastic computing introduces redundancy using ideas from error-correcting codes [21, 23, 24, 36, 39, 44, 62, 72, 75], and can let the computation continue when a preemption failure happens like ordinary error-correcting codes, the way it utilizes the coded data to deal with preemption is fundamentally different from existing works. More specifically, coded elastic computing can flexibly change the workload of each machine at runtime based on the number of available machines by selecting to use only a subset of the encoded data in a cyclic fashion. Apart from providing fault-tolerance when machines are preempted, coded elastic computing is also *positively-elastic* in that it can utilize the properties of the coded data to reduce the workload at existing machines flexibly when new machines join the computation. We will show that the coded elastic computing framework can make the computational cost at each machine scale inversely with the number of machines, which leads to linear scaling of theoretical computational cost. The proposed technique is also useful in other applications besides elastic computation when the number of machines needs to be dynamically adjusted during a learning task, such as when the number of machines is tuned as a hyper-parameter, or when the machines have to be reallocated to achieve fairness [32] between users or the specific need of some users at runtime.

The proposed coded elastic computing technique is tested in the multi-tenancy cluster at Microsoft as an example of the Apache REEF EGC (Elastic Group Communication) framework. Apache REEF is a library that helps develop distributed high-performance applications on top of cluster resource managers such as YARN [65] and Mesos [32]. The Apache REEF project provides a set of abstractions and reusable functional blocks to ease the process of building cloud-scale applications. EGC is a distributed communication framework which extends Apache REEF by providing an API allowing to implement elastic computations by chaining fault-tolerant MPI-like primitives. While fault-tolerant distributed applications are usually conducted using the well-known MapReduce-style computation model based on data shuffling [18, 48], recent trends in machine learning show that MPI-based computation [56] provides better performance with respect to the former. Nevertheless, MPI-based applications are in general not fault tolerant, whereas EGC tries to unify the benefits of the two worlds. Based on the EGC framework, we test the proposed technique for a coded implementation of linear regression on a real dataset when machines can leave and join the computation. We show that the current technique can obtain the same convergence behavior as ordinary gradient-descent-based algorithms but can elastically allocate the work load based on the number of available machines without moving data at the existing machines. We also compare with other baselines, such as ignore, replication and an existing algorithm called Elastic Distr-BGD [49] to show the improvement of the proposed technique in terms of the model generalization error.

In this paper, we first present a *coded elastic matrix-vector multiplication algorithm* to illustrate the main idea. Then, we present the generalizations of the proposed technique in broader applications, namely matrix-matrix multiplication, linear regression, and master-free fully-distributed computing. Finally, we validate the approach with a set of experiments. The contributions of the paper are thus summarized in the following:

- We formalize the preemption problem in elastic computing and propose a coded elastic data-partitioning framework to deal with it.

- We design a computing technique that can adapt the workload at each machine in the presence of elastic events, without moving data at existing machines.
- We test the proposed technique using the Apache REEF EGC framework for linear regression and show advantage on real datasets over multiple baselines.

2 Resource-Elastic Coded Distributed Computing

In this section, we formally define computation elasticity (see Section 2.1), and present coded elastic techniques. We initially focus on the problem of **matrix-vector multiplication** for having a better theoretical understanding of coded elastic computing. Before presenting the algorithm in Section 2.4, we introduce the main idea of the paper in Section 2.2 and Section 2.3. Then, in Section 2.5.1 and Section 2.5.2, we analyze the proposed techniques and prove that they are indeed elastic according to our definition.

2.1 Definition of Computation Elasticity

We consider the case when a data matrix $\mathbf{X}$ is stored distributedly at several machines. Denote by P the initial number of machines. There are *elastic events* whereby existing machines can be preempted and new machines can be added to the computation. We denote by n the number of existing machines, and m the memory cost at each machine. Note that n and m can both change over time in the elastic computing settings, and n can even become greater than P when new machines join. To characterize the key components of these elastic events, we state the following three properties:

Property 1. When a preemption failure happens, which machine(s) to be preempted is decided by the resource allocator and is not known in advance.

Property 2. The preemption is permanent, meaning that the preempted machines are going to be removed forever. However, new machines may join after an unknown time.

Property 3. After an elastic event happens, the entire system gets the information. That is, if some machines leave or join, the other machines know immediately about which machines leave or join.

The second and the third properties differentiate the elastic failures from common faults and stragglers because (1) new machines can join the computation, and (2) one may adapt the computation scheme instantly after an elastic event and possibly utilize the newly available resources. In Section 2.2 and Section 2.3, we provide the elastic data partitioning scheme to utilize resource elasticity actively.

A configuration point is a tuple (n, m) that uses n machines, with the memory cost at each machine not exceeding m . An achievable computation policy at a configuration point is said to be *optimal* if it obtains the **optimal tuple (e, u)** , where e is the number of machine preemptions that the policy can **tolerate (i.e., can continue to compute the exact result even in the presence of machine preemptions)**, and u is the size of data that a machine actually selects to use ($u \leq m$). For a configuration point (n, m) , we would like to minimize the actually selected data u to reduce the memory access and maximize the number of tolerable machine failures e to provide the best tolerance to preemptions.

For a given configuration (n, m) , i.e., n machines where each machine has memory cost m , denote by $\mathcal{A}_{n,m}$ the set of computing policies that use n machines where each machine has memory cost no more than m . Denote by $\mathcal{A}_{n,m}^* \subset \mathcal{A}_{n,m}$ the set of optimal computing policies, i.e., they obtain the optimal tuple (e^*, u^*) .

Definition 1. (transition compatibility) A pair of policies (a, a') with $a \in \mathcal{A}_{n,m}$ and $a' \in \mathcal{A}_{n',m'}$ is said to be transition compatible if the policy a can be transitioned to policy a' without having to move or modify the data in the existing machines in the event of a configuration transition $(n, m) \rightarrow (n', m')$.

The key to the above definition is that we discourage inter-machine data movement in order to make the elastic configuration transitions non-disruptive to ongoing computation tasks.

Definition 2. (fully transition compatibility) A family of policies $\mathcal{F} = \{a_{n,m}\}$, $n \in \mathcal{N}$, $m \in \mathcal{M}$, is said to be fully transition compatible if every pair $(a_{n,m}, a_{n',m'})$ in $\mathcal{F}$ are transition compatible.

Definition 3. (optimal fully transition compatibility) A family of fully transition compatible policies $\mathcal{F} = \{a_{n,m}\}$, $n \in \mathcal{N}$, $m \in \mathcal{M}$ is said to be optimal if all policies are optimal, i.e., each policy $a_{n,m} \in \mathcal{F}$ is in $\mathcal{A}_{n,m}^*$ and obtains the optimal tuple (e^*, u^*) for the number of tolerable machine preemptions and the size of the selected data to use.

Our goal is thus to find conditions under which an optimal fully transition compatible family exists, and to provide explicit characterizations of the transition compatible families. We now present matrix-vector multiplication techniques that can provide a fully transition compatible family of optimal computation policies with fixed memory cost at each machine, i.e., when m is fixed in different transition compatible policies $a_{n,m}$.

2.2 Coded Data Partitioning in the Presence of Preempted Machines

Assume that in the worst-case of preemption failures, there are at least L machines that remain². We will show that the parameter L is also equal to the *recovery threshold*, the meaning of which will be clear in this section (Section 2.2). In this paper, we consider repeatedly using the same data but with different input vectors. For matrix multiplications, **it means that we compute $\mathbf{X}\mathbf{w}_t$ for $t = 1, 2, \dots$ for the same $\mathbf{X}$.** This computation primitive is applicable in a variety of scenarios, including training linear models [29, 71], PageRank [72], model-parallel deep neural networks [19, 20] and many machine learning algorithms at the inference stage.

We partition the data matrix $\mathbf{X}$ into L subsets (or equivalently, submatrices) $\mathbf{X}_1, \mathbf{X}_2, \dots, \mathbf{X}_L$ of equal size. If the total number of data points is not divisible by L , we can use zero-padding. We generate P coded data matrices $\mathbf{X}_s^{\text{coded}}, s = 1, 2, \dots, P, (P > L)$, in which each matrix is a linear combination of the form:

$$\mathbf{X}_s^{\text{coded}} = \sum_{k=1}^L g_{s,k} \mathbf{X}_k \quad (1)$$

where each $g_{s,k}$ is a random but predetermined coefficient. The P coded data matrices are distributed to P workers. In this way, the coded data $\mathbf{X}_s^{\text{coded}}, s = 1, 2, \dots, P$ satisfy the following property with probability one for a variety of choices of the linear coefficients $g_{s,k}$, e.g., if $g_{s,k}$'s are i.i.d. Gaussian random variables:

Lemma 2.1. Suppose we want to compute the matrix-vector product $\mathbf{X}\mathbf{w}$. Then, any L out of P coded computation results $\mathbf{X}_s^{\text{coded}}\mathbf{w}, s = 1, 2, \dots, P$ are sufficient to recover the original (uncoded) computation results $\mathbf{X}_k\mathbf{w}, k = 1, 2, \dots, L$, which are equivalent to $\mathbf{X}\mathbf{w}$.

The recovery of the results is through solving L linear systems of the form $\mathbf{X}_s^{\text{coded}}\mathbf{w} = \sum_{k=1}^L g_{s,k} \mathbf{X}_k\mathbf{w}$ for the L different machines that successfully finish the computation. Lemma 2.1 is critical for the failure recovery. It essentially shows that no matter which machines are preempted, as long as the number of remaining machines is not smaller than L , **the whole information of the original data is preserved in the remaining machines. This is why we call the parameter L the recovery threshold. The parameter L is limited by the storage constraint at each machine.** The more redundancy we can add to the data, the lower recovery threshold we need, and hence more failures we can tolerate. In our experiments, we use a redundancy factor of $P/L = 2$, and hence we can at maximum tolerate failures when half of the machines are preempted. An often-used coding technique is called **systematic code, in which the linear coefficients** satisfy

$$g_{s,k} = 1_{\{s=k\}}, \text{ if } s \leq L. \quad (2)$$

In this case, the coded data at the first L machines $\mathbf{X}_s^{\text{coded}}, s = 1, 2, \dots, L$ are the original data $\mathbf{X}_k, k = 1, 2, \dots, L$. This can provide backward-compatibility to switch between coded computing and uncoded ordinary computing, and at the same time significantly reduce the cost of encoding the data at the preprocessing stage (1).

2.3 Elastic Data Partitioning for Elastic Computation by Using Data In a Cyclic Way

According to Lemma 2.1, as long as the number of machines that are not preempted is greater or equal to L , the remaining data using the coded data partitioning can preserve the whole information of the original data. However, when the number of machines is strictly larger than L , it becomes redundant to use all the coded data because the data at L machines already preserve the whole information. One may think that this amount of waste is not significant. However, consider the case when the number of machines gradually increases from L to a large number. In this case, any L machines can provide

²The parameter L , or a lower bound of L , is needed for exact computation. However, in many machine learning tasks, one can often optimize with a subset of data due to data redundancy. In that case, knowledge of L is not necessary.

the correct results, but we are not able to utilize the parallel gain if a fixed coding technique is used. The situation can be partially solved if we use some rateless coded techniques [47]. See Section 2.6 for a comparison between the proposed technique and rateless coded computing.

To positively utilize all the remaining machines and achieve the parallel computing capabilities of the extra machines, we select to use data in a cyclic fashion as shown in Figure 1. We use a systematic

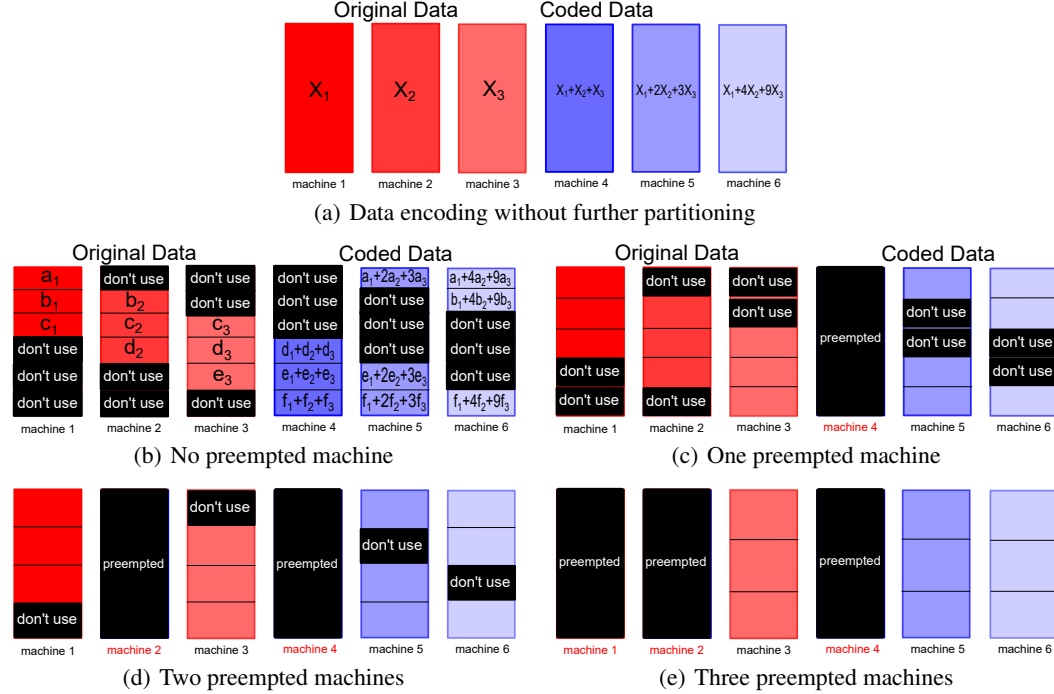


Figure 1: The main idea of elastic data partitioning is to use the data in a cyclic way. Each column of data is stored at one machine. For each group (i.e., row block) of data at different machines, there are enough number of sub-blocks that contain all the information. This cyclic way of using data leads to linear scaling of the per-machine computational cost in the number of machines.

code (see (2)) by which the first L of the P coded blocks $\mathbf{X}_s^{\text{coded}}, s = 1, 2, \dots, P$ are the original data $\mathbf{X}_k, k = 1, 2, \dots, L$. In Figure 1(a) we use red to denote original data and blue to denote the remaining coded data. In this example, the initial number of machines is $P = 6$ and the recover threshold is $L = 3$. From figure 1(b) to 1(e), we show how to continue the computation when machines are gradually preempted from 6 to 3 (machines correspond to the columns). **The stored data remains fixed i.e., the same way as in Figure 1(a), but we further partition the data into smaller blocks and only select part of the data to use.** Each machine is initially allocated a single subset of coded data $\mathbf{X}_s^{\text{coded}}, s = 1, 2, \dots, P$, among which L subsets are the original data. Each subset of data is represented as a column in any subfigure of Figure 1.

If no failures occur (see Figure 1(b)), to remove redundancy from the data, we partition each data block (column) into P sub-blocks, and let each machine only use L out of P sub-blocks. By a sub-block of data, we mean one small rectangle in Figure 1(b). If $M \geq 1$ machines are preempted (see Figure 1(c)-1(e)), we partition each data block into $P - M$ sub-blocks, and still let each node only use L out of $P - M$ sub-blocks. If new machines join, they download the coded data previously used in some failed machines or some new linearly combined data, and all the machines, including the ones that just join, use elastic data partitioning based on the current number of available machines.

There are two advantages of this type of data usage (1) the overall selected data to use is of the same size as the original data and the selected data across all remaining machines have the same size; and (2) the selected data preserve the whole information of the original data, according to Lemma 2.1. Thus, we can exactly recover the results while removing the redundancy in the way of using data. These two properties will be formally introduced and proved in Theorem 2.1.

2.4 Coded Elastic Computing for Matrix-vector Multiplications

We provide the detailed procedures of the coded elastic computing algorithm for the repeated matrix-vector multiplication problem $\mathbf{X}\mathbf{w}_t, t = 1, 2, \dots$ in Algorithm 1. We use $\mathbf{X}_{k,j}^{\text{coded}}$ to represent the j -th sub-block of the data at the k -th machine. We will call $\mathbf{X}_{k,j}^{\text{coded}}$'s with the same j “the j -th group” of sub-blocks which correspond to the j -th row block in any subfigure of Figure 1. Note that the number of row blocks change with the number of preempted machines. We use $\mathbf{G}_j$ to represent the collection of linear combination coefficients for the j -th group (row block) that are selected to use.

For example, for the first group (row block) in Figure 1(b), we have $\mathbf{G}_1 = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 2 & 3 \\ 1 & 4 & 9 \end{bmatrix}$ because the three selected sub-blocks are $\mathbf{a}_1, \mathbf{a}_1 + 2\mathbf{a}_2 + 3\mathbf{a}_3$ and $\mathbf{a}_1 + 4\mathbf{a}_2 + 9\mathbf{a}_3$. And for the last group (row block) in Figure 1(b), $\mathbf{G}_6 = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \\ 1 & 4 & 9 \end{bmatrix}$, because the three selected sub-blocks are $\mathbf{f}_1 + \mathbf{f}_2 + \mathbf{f}_3, \mathbf{f}_1 + 2\mathbf{f}_2 + 3\mathbf{f}_3$, and $\mathbf{f}_1 + 4\mathbf{f}_2 + 9\mathbf{f}_3$.

Algorithm 1 Coded Elastic Computing for Matrix-Vector Multiplication

Input: The data matrix $\mathbf{X}$, the number of machines P , the recovery threshold L , the linear combination coefficients $g_{s,k}$'s in equation (1) and the sequence of input vectors $\mathbf{w}_t, t = 1, 2, \dots$

Preprocessing: Partition the data $\mathbf{X}$ into L subsets and compute the coded subsets as in (1).

Online computation:

FOR each computation with input $\mathbf{w}_t$:

Broadcast: The master node sends $\mathbf{w}_t$ to each worker.

FOR each group index j :

Gather: The k -th worker computes $\mathbf{u}_{t,k,j} = \mathbf{X}_{k,j}^{\text{coded}}\mathbf{w}_t$ and sends $\mathbf{u}_{t,k,j}$ to the master.

 The master gathers vectors $\mathbf{u}_{t,k,j}$ for all workers that use the j -th sub-block and obtains the matrix $\mathbf{u}_{t,j}$ which contains the results for the j -th group (row block).

Decode: The master node computes $\mathbf{u}_{t,j}\mathbf{G}_j^{-1}$ to obtain the results for the j -th group.

Output: The master node outputs $\mathbf{X}\mathbf{w}_t$.

IF Preemption/New Machines: Change the selected data to use based on the current number of machines.

2.5 Analysis of Coded Elastic Computing: Achieving Optimal Fully Transition Compatibility

According to Definition 3, a fully compatible family of policies can support seamless transitions between any pairs of policies in the family. The coded elastic computing scheme provided in Algorithm 1 for matrix-vector multiplication gives a family of fully-compatible policies for fixed memory cost at each machine. In Section 2.5.1, we analyze the memory cost and the number of data points to be used at each existing machine. Then, in Section 2.5.2, we provide lower bounds on these two quantities and show that Algorithm 1 achieves the optimal memory cost and the size of the selected data. Thus, the coded computing policies in Algorithm 1 is a compatible family of *optimal* policies, in that for the fixed storage size, each policy obtains the largest number of tolerable failures and smallest size of the selected data to use (e^*, u^*), and provides seamless transitions between each other without moving data at existing machines.

2.5.1 Upper Bounds on the Storage Cost and the Size of the Selected Data to Use

Suppose the original data has N data points and all data points are in $\mathbb{R}^d$. In the following theorem, by the *size* of the data we mean the number of data points.

Theorem 2.1. The coded elastic computing algorithm achieves the exact computation result of $\mathbf{X}\mathbf{w}_t$ for all t . The size of the data stored at each machine is N/L . The size of the selected data to use at each machine is $N/(P - M)$ and is the same across different machines. The overall size of the selected data is the same as the size of the original data.

Proof. Recall that we call the sub-blocks on the same row block (in Figure 1) of the P different blocks of data a group, and we use $\mathbf{X}_{k,j}^{\text{coded}}$ to represent the coded sub-block of data that is at the k -th

machine and belongs to the j -th group. Let $\mathbf{X}^j$ be the collections of original data that belongs to the j -th group. For example, in Figure 1(b), $\mathbf{X}^6$ represents the collection of original data $[\mathbf{f}_1, \mathbf{f}_2, \mathbf{f}_3]^\top$.

Then, from the cyclic way of using data, we can see that when each machine uses L sub-blocks, the overall number of used sub-blocks in each group is L . From Lemma 2.1, the results $\mathbf{u}_{t,k,j} = \mathbf{X}_{k,j}^{\text{coded}} \mathbf{w}_t$ for all the workers that use the coded data in the j -th group can be collected together to decode $\mathbf{X}^j \mathbf{w}_t$.

The other claim can be seen from the figure, i.e., the area of the used data is always equal to the area of the original data, and the area of the used data is the same across all remaining machines. We prove this claim as follows. The size of the data in each of the L subsets is N/L . Thus, the used data at each machine is the same number $\frac{N}{L} \frac{L}{P-M} = N/(P-M)$. There are $P-M$ machines left, so the overall size of the used data is $\frac{N}{P-M} \cdot (P-M) = N$, which is the same as the original data. $\square$

Theorem 2.1 shows that our technique uses the same size of data as the original (uncoded) case. This is desirable for memory-bound applications.

Remark 1. (Cost analysis) Recall that P is the number of workers, $N \times d$ is the size of the matrix $\mathbf{X}$, L is the recovery threshold, and $P-M$ is the number of currently available machines. The encoding (preprocessing step) is a one-time cost for online matrix-vector multiplications. The decoding by solving a linear system at the master node has computational cost $\mathcal{O}(LN)$, because the linear system for each group (row-block) of data involves L equations on L unknown subvectors of size $N/L/(P-M)$ (which is the height of each sub-block), and there are $P-M$ such groups. Thus, the computational cost using straightforward matrix-vector multiplication is $L^2 \cdot N/L/(P-M) \cdot (P-M) = NL$. The matrix-multiplication step at each worker has cost $\mathcal{O}(dN/(P-M))$. Thus, the decoding cost is smaller than the computational cost at each worker as long as $d = \Omega((P-M)L)$. Even if $d < \Omega((P-M)L)$, we can partition the machines into smaller groups and respectively code each group. The decoding complexity can be further reduced to $\mathcal{O}(N \log^2 L)$ if Vandermonde systems are used [42], at the cost of numerical stability. One thing to note is that the decoding complexity, even for the straightforward matrix-vector multiplication method, is NL and is independent of the number of workers P .

2.5.2 Lower Bounds on the Storage Cost and the Size of the Selected Data to Use

Here, we provide a fundamental limit which shows that the achievable scheme provided in 2.1 is optimal in terms of the storage cost at each machine and the size of the actually used data at each iteration, for a fixed number of machines and a fixed number of tolerable machine preemptions. Before we present the theorem, we formalize the definition of the *size* of data using number of bits. This is because in theory, we cannot store arbitrarily high-precision numbers.

Assumption 1. Suppose the entries of the matrix $\mathbf{X}$ are i.i.d. random variables that take values in a finite set $\mathcal{S} \in \mathbb{R}$. Each of this random variable has entropy $H = \log |\mathcal{S}|$. Thus, the overall entropy of all the data in $\mathbf{X}$ is NdH .

Assumption 2. For a certain computation policy, suppose each machine initially stores an array of finite-precision numbers in its memory. Each finite-precision number can be an *arbitrary* function of the original data. The overall number of finite-precision numbers that is stored is finite.

Note that although we use real-number computation all the time, the numbers that we deal with are always discrete, i.e., we conduct computation of finite-precision numbers in the real field. This validates Assumption 1. Also note that the number of possible combinations of floating point numbers that can be stored by a finite-length bit array is finite. Therefore, Assumption 2 is also valid. We need these two assumptions because if they do not hold, the system can concatenate all the real numbers in $\mathbf{X}$ into one real number and only stores that particular real number. In that case, no bound on the storage is meaningful because one only needs to read this single number in memory to access all the information of $\mathbf{X}$. We also need the assumption that the size of the stored numbers are finite because otherwise, we can enumerate all possible $\mathbf{X}\mathbf{w}$ and store them.

Assumption 3. Suppose we do not alter the way that we store the data $\mathbf{X}$ inside the memory after the computation begins. Even when a preemption-type failure happens, we do not move the data at existing machines.

Assumption 4. By *selecting to use* one number stored in the memory, we mean the algorithm reads the whole number (e.g., reading all digits if the number is stored as a floating-point number) from the memory for further processing. The array of finite-precision numbers can only be accessed one

number at a time, meaning that one cannot access a function value of two numbers and claim that only one number is selected to use.

Theorem 2.2. Suppose the Assumptions 1-4 hold. Suppose we require the recovery of the exact computation result $\mathbf{X}\mathbf{w}$. Then, the following fundamental limits hold.

- (a) Denote the entropy of the encoded data at the k -th machine by $H_k, k = 1, 2, \dots, P$. Then, to provide the tolerance to a maximum of $P - L$ failures, we have $\max_{k \in \{1, 2, \dots, P\}} H_k \geq \frac{N}{L} \cdot dH$;
- (b) The worst-case entropy of the actually used data (maximized with respect to the choice of $\mathbf{w}$) has to be no less than NdH , or $N/(P - M) \cdot dH$ at each machine.

Remark 2. By comparing Theorem 2.1 and Theorem 2.2, we see that the coded elastic computing technique in Algorithm 1 achieves the fundamental limit because each data point has dimension d and each entry has entropy H . We note a nuance here that the linear combinations in Algorithm 1 may make each encoded number have entropy larger than H . However, this can be solved if (i) we approximate the real-number computation using the computations in a finite field and assume each number in the data matrix $\mathbf{X}$ has a uniform distribution over the finite field [75], or (ii) we assume that each entry in the data matrix comes from a quantized version of the standard Gaussian-distribution, and assume that the encoding matrix has normalized rows, such that the linear combination result in (1) (before quantization) is also from the standard Gaussian distribution. To make the point (ii) formal, suppose an arbitrary entry in the data matrix $\mathbf{X}_k$ is sampled from i.i.d. standard Gaussian and quantized to $q(y_k)$ (here $y_k \sim \mathcal{N}(0, 1)$). A linear combination of the samples has the form $\sum_{k=1}^L g_{s,k} q(y_k)$ (see equation (1)), where the linear combination coefficients satisfies $\sum_{k=1}^L g_{s,k}^2 = 1$. Then, the linear combination is stored as $q(\sum_{k=1}^L g_{s,k} q(y_k))$ in the s -th machine. Since the *unquantized* sum $\sum_{k=1}^L g_{s,k} y_k$ is a standard Gaussian random variable, its quantized version $q(\sum_{k=1}^L g_{s,k} y_k)$ has the same entropy as each $q(y_k)$. Therefore, we only need to show that $q(\sum_{k=1}^L g_{s,k} q(y_k))$ has an entropy that is close to $q(\sum_{k=1}^L g_{s,k} y_k)$. This can be proved by bounding the difference in the entropy, because the two random variables $q(\sum_{k=1}^L g_{s,k} q(y_k))$ and $q(\sum_{k=1}^L g_{s,k} y_k)$ have the same support, and their pmf's can be made arbitrarily close when the quantization function q has a quantization level $\Delta \rightarrow 0$.

Remark 3. Note that the claim (b) has to be stated in a worst-case way because for many choices of $\mathbf{w}$, computing $\mathbf{X}\mathbf{w}$ can be degenerated. For example, if we know in advance that $\mathbf{w}$ only takes value in a very small finite set of vectors, we can compute $\mathbf{X}\mathbf{w}$ for all possible $\mathbf{w}$ and store these vectors. When $\mathbf{w}$ is sparse, we also do not need to read the entire matrix $\mathbf{X}$. Therefore, we indeed need to state the fundamental lower bound in terms of the worst-case $\mathbf{w}$.

Proof. Now, we prove statement (a). Suppose an arbitrary set of $P - L$ machines fail. Since the algorithm is tolerant to any $P - L$ failures, the master node can still recover exactly all the results $\mathbf{X}\mathbf{w}$, no matter what $\mathbf{w}$ is. Therefore, we can choose $\mathbf{w}$ to be the elements of natural basis $\mathbf{w} = \mathbf{e}_i, i = 1, 2, \dots, d$, and collect all the results $[\mathbf{X}\mathbf{e}_1, \mathbf{X}\mathbf{e}_2, \dots, \mathbf{X}\mathbf{e}_d] = \mathbf{X}\mathbf{I}_d = \mathbf{X}$. Since data processing can only reduce entropy (from the data processing inequality), the entropy of the overall data stored at the remaining L machines is no less than the entropy of $\mathbf{X}$ which is NdH . This holds for any combination of L machines, i.e.,

$$H(\mathbf{X}_{i_1}, \mathbf{X}_{i_2}, \dots, \mathbf{X}_{i_L}) \geq NdH, \forall 1 \leq i_1 < i_2 < \dots < i_L \leq P. \quad (3)$$

Adding up the above equation for all combinations of L out of P machines, and by plugging in $\sum_{j=1}^L H(\mathbf{X}_{i_j}) \geq H(\mathbf{X}_{i_1}, \mathbf{X}_{i_2}, \dots, \mathbf{X}_{i_L})$, we have $\sum_{k=1}^P H_k \geq \frac{NPdH}{L}$.

Then, we prove statement (b). Suppose (b) is not true. Then, it means that there exists a way to encode and store the encoded data in the memory of the machines, such that for any arbitrary vector $\mathbf{w}$, computing $\mathbf{X}\mathbf{w}$ only requires reading data of entropy strictly less than NdH . Since the overall number of stored data is finite, we can assume that the overall number of stored numbers is p . For an arbitrary subset $\mathcal{S}$ of the stored numbers such that $H(\mathcal{S}) < NdH$, we denote by $\mathcal{P}_{\mathcal{S}}$ the set of $\mathbf{w}$ such that $\mathbf{X}\mathbf{w}$ is able to be computed using only the numbers in $\mathcal{S}$. Then, we can see that the $\mathcal{P}_{\mathcal{S}}$ for an arbitrary subset $\mathcal{S}$ is a linear subspace of $\mathbb{R}^d$. This is because if $\mathbf{X}\mathbf{w}_1$ and $\mathbf{X}\mathbf{w}_2$ are both able to be computed using the numbers in $\mathcal{S}$, then, $a\mathbf{X}\mathbf{w}_1 + b\mathbf{X}\mathbf{w}_2 = \mathbf{X}(a\mathbf{w}_1 + b\mathbf{w}_2)$ is also able to be computed. Now, we prove that $\mathcal{P}_{\mathcal{S}}$ cannot be the entire $\mathbb{R}^d$ that $\mathbf{w}$ can take value from. This is

because if $\mathcal{P}_S$ is equal to $\mathbb{R}^d$, then $\mathbf{X}\mathbf{e}_1, \mathbf{X}\mathbf{e}_2, \dots, \mathbf{X}\mathbf{e}_d$ are all able to be computed using the numbers in $\mathcal{S}$ ($\mathbf{e}_1, \dots, \mathbf{e}_d$ are the standard basis), which means $\mathbf{X}$ itself is able to be computed using $\mathcal{S}$. This is a clear contradiction to the data-processing inequality because $H(\mathbf{X}) = NdH$, while $H(\mathcal{S}) < NdH$. Thus, $\mathcal{P}_S$ can at most be a linear space of dimension $d - 1$ in $\mathbb{R}^d$. This means the union of $\mathcal{P}_S$ for all $\mathcal{S}$ is a finite collection of linear spaces of dimension $d - 1$ in $\mathbb{R}^d$, which cannot cover the entire $\mathbb{R}^d$. Thus, there must exist $\mathbf{w}$ in $\mathbb{R}^d$ such that $\mathbf{X}\mathbf{w}$ is not able to be computed by using data of entropy strictly less than NdH . $\square$

2.6 Related Works and the Comparison with Non-elastic Coded Computing Techniques

Coded computing is an emerging area to tackle stragglers [8, 11, 21–26, 36, 37, 39–41, 46, 47, 50, 51, 55, 57, 58, 60, 63, 69, 72, 76, 77], machine failures and soft errors [9, 20, 30, 35, 73], security issues and adversaries [13–16, 70, 77], and communication bottlenecks [28, 31, 34, 34, 44, 53, 54, 62, 74]. It can handle both linear and nonlinear computations [20, 38, 62]. It is a significant advance on classical algorithm-based fault-tolerance (ABFT) techniques [33] and noisy computing [52, 64, 66], and often achieves scaling sense improvements. Our work is the first to address the system elasticity issue using coded computing.

Non-elastic coded computing cannot adapt the algorithmic procedures to exploit *new* machines. We consider the case when the number of machines increases from L (i.e., the recovery threshold) to infinity with fixed storage size at each machine. We can easily see that elastic coded computing can have lower and lower memory-access time by using less and less data at each machine, while ordinary coded computing cannot because the used data remains unchanged at each machine. However, each node can have more than one linear combinations [47]. E.g., suppose $L=8$ linear combinations suffice to recover the result, and each machine has 8 linear combinations. Then, when the number of machines increases from 1 to 8, rateless coded computing can move between the optimal points $(n, m) = (1, 8), (2, 8), (4, 8), (8, 8)$, i.e., it achieves the optimal size of the selected data to use at some of the (n, m) configurations. But they cannot move beyond the point $(8, 8)$ and use more machines, because each machine, in the rateless-coded case, cannot use smaller than 1 (a fraction) of linear combinations. From another perspective, for the rateless-coded scheme to achieve fully transition compatibility in the large scale, the number of linear combinations at each machine also has to grow with the number of (possible) machines, leading to scaling-sense higher decoding complexity than the elastic partitioning technique in Algorithm 1 (Note that the decoding cost in Algorithm 1 does not increase with the number of machines, and can be further reduced if parallel decoding is allowed. See Section 3.3 for details). Another advantage of coded elastic computing compared to the rateless-coded scheme is that it allows non-uniform partitioning of each row-strip (see Figure 1) and can have much more fine-grained task scheduling. Regarding the communication efficiency of coded elastic computing, see Section 3.3 for a fully-distributed version of coded elastic computing.

For task-level failures or stragglers, dynamic task allocation is useful [67, 68]. The task scheduler can choose which tasks to replicate, relaunch based on task profiles, and delay the relaunching to save time [7, 8, 10, 27, 61]. However, unlike task relaunching and allocation, recovering from machine preemptions requires both restoring the machine states and downloading the data, which is time-consuming. The situation gets worse if elastic events are frequent.

3 Applications of the Coded Elastic Computing

The cyclic way of elastic data partitioning applies to general coded computing techniques proposed thus far and is not limited to matrix-vector multiplications.

3.1 Matrix-Matrix Multiplications

First, we consider the application of matrix-matrix multiplications. We consider an *online* version: we store an encoded version of the matrix $\mathbf{A}$ at P machines and compute the matrix multiplication $\mathbf{A}\mathbf{B}$ for different $\mathbf{B}$'s. It is shown [23, 24, 76] that the storage-optimal technique for coded matrix multiplications is the MatDot scheme proposed in [23, 24]. This technique partitions the matrix $\mathbf{A}$ column-wise and the matrix $\mathbf{B}$ row-wise, and stores linearly combined submatrices of $\mathbf{A}$ and $\mathbf{B}$ at each machine. In an online setting with elastic machine preemptions, we encode and store $\mathbf{A}$ at the initial stage, and do not move $\mathbf{A}$ anymore. When we receive $\mathbf{B}$, we still partition the matrix $\mathbf{B}$

row-wise but linearly combine them using the knowledge of the availability of the machines. The advantage is that we do not need to use the polynomial-based codes and can thus avoid possible numerical issues. At the same time, since the availability of the machines are known before we encode $\mathbf{B}$, we can also remove the factor of 2 in the recovery threshold of MatDot codes. We can also use the same computational time cost as the uncoded case, which is similar to what we can achieve in the matrix-vector case.

More precisely, suppose we partition the matrix $\mathbf{A}$ into L column blocks $\mathbf{A}_1, \dots, \mathbf{A}_L$, and encode these blocks into P blocks $\mathbf{A}_s^{\text{coded}}, s = 1, 2, \dots, P$, where P is the initial number of machines:

$$\mathbf{A}_s^{\text{coded}} = \sum_{k=1}^L g_{s,k} \mathbf{A}_k. \quad (4)$$

Denote by $\mathbf{G}$ the matrix $[g_{s,k}]$, which is of size $P \times L$. Assume M machines are preempted and $P - M \geq L$ machines remain. Similar to Algorithm 1, we partition each coded sub-matrix $\mathbf{A}_s^{\text{coded}}$ into $P - M$ submatrices. However, here, we partition each $\mathbf{A}_s^{\text{coded}}$ *column-wise* to $\mathbf{A}_{s,i}^{\text{coded}}, i = 1, 2, \dots, P - M$. This is mathematically equivalent to partitioning each uncoded submatrix $\mathbf{A}_s$ into $\mathbf{A}_{s,i}, i = 1, 2, \dots, P - M$, and compute

$$\mathbf{A}_{s,i}^{\text{coded}} = \sum_{k=1}^L g_{s,k} \mathbf{A}_{k,i}, i = 1, 2, \dots, P - M. \quad (5)$$

The subscript s in $\mathbf{A}_{s,i}$ belongs to a subset with size $P - M$ of the set $\{1, 2, \dots, P\}$, which corresponds to the $P - M$ available machines after the preemption failures. We again select to use these submatrices $\mathbf{A}_{s,i}$ in a cyclic fashion, just as shown in Figure 1. For example, consider the case when $P = 6, L = 3, M = 2$ and the 2nd and the 4th machines are preempted, which is exactly the same as shown in Figure 1(d). Then, we use $\mathbf{A}_{1,1}, \mathbf{A}_{1,2}, \mathbf{A}_{1,3}$ at the 1st machine, $\mathbf{A}_{3,2}, \mathbf{A}_{3,3}, \mathbf{A}_{3,4}$ at the 3rd machine, and so on. Denote by $\mathcal{S}_i$ the set of the indices of all the machines that use $\mathbf{A}_{s,i}$. For example, for $i = 1$, in the above example, $\mathcal{S}_1 = \{1, 5, 6\}$. Again, similar to Section 2.4, denote by $\mathbf{G}_i$ the submatrix of $\mathbf{G}$ with row indices in $\mathcal{S}_i$. Each $\mathbf{G}_i$ is a $L \times L$ matrix. Denote by

$$\mathbf{H}_i = (\mathbf{G}_i^\top)^{-1}, \quad (6)$$

and assume $\mathbf{H}_i$ has element $h_{l,k}, l = 1, 2, \dots, L, k = 1, 2, \dots, L$.

When we get the matrix $\mathbf{B}$, we partition it row-wise into L blocks $\mathbf{B}_1, \mathbf{B}_2, \dots, \mathbf{B}_L$. Then, we partition each submatrix $\mathbf{B}_k$ into $P - M$ row blocks $\mathbf{B}_{k,i}, i = 1, 2, \dots, P - M$ as well. For each i , suppose $\mathcal{S}_i = \{s_{i,1}, s_{i,2}, \dots, s_{i,L}\}$, where $1 \leq s_{i,1} < s_{i,2} < \dots < s_{i,L} \leq P$. Then, we encode

$$\mathbf{B}_{s_{i,l},i}^{\text{coded}} = \sum_{k=1}^L h_{l,k} \mathbf{B}_{k,i}, \quad (7)$$

and send $\mathbf{B}_{s_{i,l},i}^{\text{coded}}$ to the $s_{i,l}$ -th machine to compute $\mathbf{A}_{s_{i,l},i}^{\text{coded}} \mathbf{B}_{s_{i,l},i}^{\text{coded}}$.

Now, we show that when we do a reduction on all the partial results $\mathbf{A}_{s_{i,l},i}^{\text{coded}} \mathbf{B}_{s_{i,l},i}^{\text{coded}}$, we can indeed get $\mathbf{AB}$.

Lemma 3.1. If we use the cyclic partitioning technique to determine each $\mathcal{S}_i = \{s_{i,1}, s_{i,2}, \dots, s_{i,L}\}$ and encode $\mathbf{B}$ according to (7), we have

$$\sum_{i=1}^{P-M} \sum_{l=1}^L \mathbf{A}_{s_{i,l},i}^{\text{coded}} \mathbf{B}_{s_{i,l},i}^{\text{coded}} = \mathbf{AB}. \quad (8)$$

Proof. Note that

$$\begin{aligned} \mathbf{A}_{s_{i,l},i}^{\text{coded}} \mathbf{B}_{s_{i,l},i}^{\text{coded}} &= \left(\sum_{k=1}^L g_{s_{i,l},k} \mathbf{A}_{k,i} \right) \left(\sum_{k=1}^L h_{l,k} \mathbf{B}_{k,i} \right) \\ &\stackrel{(a)}{=} [\mathbf{A}_{1,i}, \mathbf{A}_{2,i}, \dots, \mathbf{A}_{L,i}] (\mathbf{G}_i)_{l\text{-th row}}^\top (\mathbf{H}_i)_{l\text{-th row}} [\mathbf{B}_{1,i}^\top, \mathbf{B}_{2,i}^\top, \dots, \mathbf{B}_{L,i}^\top]^\top, \end{aligned} \quad (9)$$

where (a) holds because $[g_{s_{i,l},1}, g_{s_{i,l},2}, \dots, g_{s_{i,l},L}]$ is the $s_{i,l}$ -th row in $\mathbf{G}$, which is the l -th row in $\mathbf{G}_i$ (recall that $\mathbf{G}_i$ is the submatrix of $\mathbf{G}$ with row indices in $\mathcal{S}_i = \{s_{i,1}, s_{i,2}, \dots, s_{i,L}\}$). Thus, by adding up the above equation for all l , we have that for each i ,

$$\begin{aligned} \sum_{l=1}^L \mathbf{A}_{s_{i,l},i}^{\text{coded}} \mathbf{B}_{s_{i,l},i}^{\text{coded}} &= \sum_{l=1}^L [\mathbf{A}_{1,i}, \mathbf{A}_{2,i}, \dots, \mathbf{A}_{L,i}] (\mathbf{G}_i)_{l\text{-th row}}^\top (\mathbf{H}_i)_{l\text{-th row}} [\mathbf{B}_{1,i}^\top, \mathbf{B}_{2,i}^\top, \dots, \mathbf{B}_{L,i}^\top]^\top \\ &= [\mathbf{A}_{1,i}, \mathbf{A}_{2,i}, \dots, \mathbf{A}_{L,i}] \mathbf{G}_i^\top \mathbf{H}_i [\mathbf{B}_{1,i}^\top, \mathbf{B}_{2,i}^\top, \dots, \mathbf{B}_{L,i}^\top]^\top \\ &\stackrel{(a)}{=} [\mathbf{A}_{1,i}, \mathbf{A}_{2,i}, \dots, \mathbf{A}_{L,i}] [\mathbf{B}_{1,i}^\top, \mathbf{B}_{2,i}^\top, \dots, \mathbf{B}_{L,i}^\top]^\top \\ &= \sum_{k=1}^L \mathbf{A}_{k,i} \mathbf{B}_{k,i}, \end{aligned} \tag{10}$$

where the equation (a) holds because $\mathbf{H}_i = (\mathbf{G}_i^\top)^{-1}$ (see equation (6)). Adding up the above equation for all i , we have

$$\sum_{i=1}^{P-M} \sum_{l=1}^L \mathbf{A}_{s_{i,l},i}^{\text{coded}} \mathbf{B}_{s_{i,l},i}^{\text{coded}} = \sum_{i=1}^{P-M} \sum_{k=1}^L \mathbf{A}_{k,i} \mathbf{B}_{k,i} = \mathbf{A} \mathbf{B}. \tag{11}$$

□

Remark 4. (Cost analysis) We can see that the computational cost for this algorithm comes from two parts: (1) the encoding of $\mathbf{B}$ using (7), and (2) the computational cost of $\mathbf{A}_{s_{i,l},i}^{\text{coded}} \mathbf{B}_{s_{i,l},i}^{\text{coded}}$. Assume the matrix $\mathbf{A}$ has size $d_A \times N$ and $\mathbf{B}$ has size $N \times d_B$, where $d_A, d_B = \Theta(N)$. Then, encoding $\mathbf{B}$ has complexity $(P-M)L \cdot L \cdot (d_B \times N/L/(P-M)) = Ld_B N = \Theta(LN^2)$. Computation of a single $\mathbf{A}_{s_{i,l},i}^{\text{coded}} \mathbf{B}_{s_{i,l},i}^{\text{coded}}$ has complexity $d_A \times N/L/(P-M) \times d_B$, and each machine computes L of them, which means the overall complexity is $d_A \times N/L/(P-M) \times d_B \times L = d_A d_B N/(P-M)$. Note that this is in the order of $\Theta(N^3)$ and the complexity is the same as distributing the matrix-matrix multiplication task to $P-M$ machines. The encoding time is much less than the computation time per worker if $Ld_B N \ll d_A d_B N/(P-M)$, or $L(P-M) \ll d_A$. If the encoding time is much smaller than the matrix-multiplication time, we again achieve linear scaling of the theoretical computational complexity in the number of available machines $P-M$ without moving data at the existing machines. At the same time, the computation time cost is the same at each worker machine as the uncoded case, which means that the factor 2 in the MatDot codes [23] can be removed³.

3.2 Coded Elastic Computing for Linear Models

Then, we focus on the application of coded computing for linear regression [39, 43]. For the ease of presentation, we consider vanilla gradient descent (we also use line search in the experiment validation for all competing techniques), in which the full matrix $\mathbf{X}$ is used at each iteration. The technique developed here naturally generalizes to stochastic gradient descent and other generalized linear models such as logistic regression. Consider the linear objective function:

$$f(\mathbf{w}; \mathbf{X}, \mathbf{y}) = \sum_{i=1}^n (\mathbf{w}^\top \mathbf{x}_i - \mathbf{y}_i)^2 + h(\mathbf{w}). \tag{12}$$

The vanilla distributed gradient descent has the form $\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \mathbf{g}_t$ and $\mathbf{g}_t = \mathbf{X}^\top (\mathbf{X} \mathbf{w}_t - \mathbf{y}) + \partial_{\mathbf{w}} h(\mathbf{w}_t)$. When the data matrix $\mathbf{X}$ is large, the most time-consuming part is the computation of $\mathbf{X}^\top (\mathbf{X} \mathbf{w}_t - \mathbf{y})$. We thus extend Algorithm 1 in the following way to compute $\mathbf{X}^\top (\mathbf{X} \mathbf{w}_t - \mathbf{y})$. It is nothing but a combination of Algorithm 1 and the matrix-matrix multiplication algorithm in Section 3.1.

- Compute $\mathbf{X}^j \mathbf{w}_t$ (where $\mathbf{X}^j$ is the data in the j -th group, or the j -th row block in Figure 1 across different machines) for all group-index j in an elastic way using Algorithm 1;
- The master computes $\mathbf{z}_t^j = \mathbf{X}^j \mathbf{w}_t - \mathbf{y}^j$ for all group-index j , where $\mathbf{y}^j$ are the labels corresponding to the data points in the j -th group;

³It was shown that the optimal recovery threshold in MatDot is $2L-1$, which means that the computational cost at each machine is twice that of the uncoded case for the same number of machines. The scheme here removes the factor of 2 because we consider a different problem: the system becomes aware of the indices of the failed machines after the preemptions have happened, and can adaptively change the encoding of $\mathbf{B}$.

- The master re-encodes $\mathbf{z}_t^j$'s using the (pre-computed) inverse generator matrix $\mathbf{H}_j = (\mathbf{G}_j^{-1})^\top$ to obtain $(\mathbf{G}_j^{-1})^\top \mathbf{z}_t^j$, and scatters the results to the workers that use the j -th group of data;
- Since data at workers are encoded using $\mathbf{G}_j$, the reduced results from all the workers are

$$\sum_j (\mathbf{X}^j)^\top \mathbf{G}_j^\top (\mathbf{G}_j^{-1})^\top \mathbf{z}_t^j = \sum_j (\mathbf{X}^j)^\top (\mathbf{X}^j \mathbf{w}_t - \mathbf{y}^j) = \mathbf{X}^\top (\mathbf{X} \mathbf{w}_t - \mathbf{y}). \quad (13)$$

Note that in the above extension of Algorithm 1, the workers also utilize the data as in Figure 1. The experiment results of coded elastic computing in linear models are provided in Section 4.

3.3 Fully Distributed Coded Elastic Computing

Coded elastic computing is not restricted to a master-worker setting. In this section, we consider a fully distributed coded elastic matrix-vector multiplication technique that is a trivial generalization of Algorithm 1. The advantage of a fully distributed framework is that communication among workers can be overlapped and the communication to the master does not become the single bottleneck in the limit of a large number of machines. Consider an application of iterative matrix-vector multiplication

$$\mathbf{w}_{t+1} = f(\mathbf{X} \mathbf{w}_t), \quad (14)$$

where $\mathbf{X}$ is a square data matrix, and $f(\cdot)$ is an entry-wise low-complexity operation on the vector $\mathbf{X} \mathbf{w}_t$. We again consider the example shown in Figure 1,⁴ but apart from the 6 worker machines, there is no master-node. In each iteration, each machine computes its own matrix-vector multiplication based on the selected data to use. For example, the 2nd machine computes $\mathbf{b}_2 \mathbf{w}_t$, $\mathbf{c}_2 \mathbf{w}_t$, $\mathbf{d}_2 \mathbf{w}_t$, and stacks the results into one vector. Then, the results at each machine can be broadcast to all the other machines using an all-gather communication (using a bucketing algorithm [12]). The communication is also done in a cyclic way so that all the communications can be maximally overlapped (i.e., all machines can communicate at the same time). In this way, the communication bandwidth can be reduced when compared to the master-worker framework. Moreover, the overall communication time in this scheme is the same as the uncoded scheme using the same number of machines, because the selected size of data at each machine is the same as the uncoded case. The decoding of the intermediate results at each iteration is done independently at each machine. The decoding can also be conducted distributedly for each group of data (each row-block), i.e., each machine only takes care of the decoding of each row-block. This can lead to increase on the communication time, but can significantly reduce the decoding time. Also, the smaller communication is possible at the cost of more storage or computation [6, 44, 59, 74] in the individual machines.

4 Implementation and Experimental Evaluation

The proposed coded elastic computing technique has been implemented on top of Apache REEF [17] Elastic Group Communication (EGC) framework⁵. REEF EGC provides an API allowing to implement elastic computations by chaining fault-tolerant MPI-like primitives. In this short paper we assess the performance of our elastic code computing approach through 2 mini-benchmarks.

Matrix-vector mini-benchmark. In this mini-benchmark, we test that indeed the time cost decreases linearly with the increase in the number of machines available. We mimic an elastic computing environment on Amazon EC2 by using different numbers of t2.large instances to compute the same matrix-vector product $\mathbf{X} \mathbf{w}$. The matrix is randomly generated and with size 30000×10000 , and it is partitioned initially into 3 submatrices of size 10000×10000 . Then, they are encoded into 6 submatrices of the same size, and each submatrix is stored at one machine (for a total of 6 machines). To mimic the elastic events, we change the number of available machines by injecting artificial failures. The maximum number of failures is 3. The per-iteration overall time (including both communication and computation) is shown in Figure 2(a). The result is averaged using 20 independent trials. As we can see, the coded elastic computing technique can utilize the extra machines when the number of machines increases.

⁴Note that the example in Figure 1 partitions the data row-wise. The column-wise partition can use the elastic coding scheme in Section 3.1, which is essentially an elastic dot-product scheme.

⁵<https://github.com/interesaaat/reef/tree/elastic-sync>

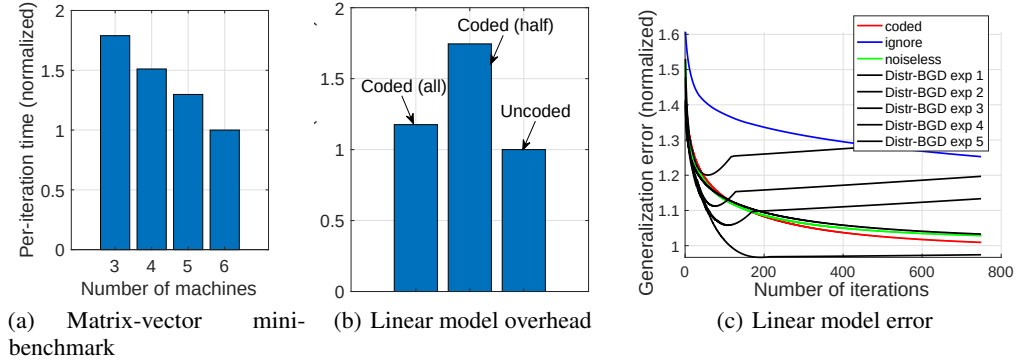


Figure 2: Mini-benchmarks experiments (results normalized due to confidentiality).

Linear model mini-benchmark. In this experiment, we test a coded implementation of linear regression using line-search-based batch gradient descent (the same setting as the baseline [49]). We run the test over 20 machines on a Microsoft internal multi-tenancy cluster. Each data point in the dataset has 3352 features, and we sample 10000 data for training and 10000 data for testing. We generate random failures and allow REEF EGC to reschedule new machines when failures occur. We start with Figure 2(b) where we plot the time for each iteration. In theory, when all the workers are present, the computational cost per iteration should be the same as the uncoded case. However, the coded method (all) has slight overhead due to decoding cost. The coded method (half) shows the cost when only half of the workers are running, which is, as expected, twice the cost of the uncoded method. In Figure 2(c) we report the generalization error and we compare our coded elastic computing technique with three baselines, namely noiseless (no failure), ignore the failure and continue, and an existing algorithm called Elastic Distr-BGD [49]. The coded method can achieve the same convergence behavior as the noiseless case, while the ignore method achieves worst generalization error. In the figure, we show 5 different experiments on Distr-BGD using the same failure probability but different realizations. The convergence of Distr-BGD depends on when a failure occurs and can lead to different algorithm performance. This is because the Distr-BGD keeps using previous gradient vectors at the failed machines, and this can (1) lead to overfitting, and (2) make the optimization miss the minimum point. In the plot of Distr-BGD, the *valley* part is due to overfitting, and the sudden change to near flat loss growth is because when the gradient descent has missed the optimal point of empirical training loss, the fixed gradient at the failure nodes makes the line search choose the smallest step size. In some cases, the Distr-BGD works extremely well because the fixed gradients act like momentum and can improve the speed of convergence.

From the experiment results, we can see that the coded elastic computing technique can obtain the same convergence behavior as ordinary gradient-descent-based algorithms but can elastically allocate the workload based on the number of available machines without moving data around.

5 Conclusions

The coded elastic computing framework presented in this paper can deal with new cloud offerings where machines can leave and join during the computation. Our framework handles the elastic events in a positive way, meaning that when machines leave, it shifts the computation to the remaining workers, and when new machines join the computation, it actively reduces the workload of existing machines without the reallocation of data. We prove that the coded elastic computing technique can achieve the same memory-access cost as the noiseless case, and hence is optimal for memory-bound applications. Using experiments in both Amazon EC2 and on a Microsoft multi-tenancy cluster, we show that the coded elastic computing technique can achieve the same convergence behavior as if no failure occurs, and can dynamically adjust working loads respect to the number of remaining workers. The proposed technique can be applied to coded matrix-vector, matrix-matrix multiplications and linear regression, and potentially other applications where the large-scale matrix operations are the bottleneck.

References

- [1] AWS Spot Instances. <https://aws.amazon.com/ec2/spot/>, 2018.
- [2] AWS Spot Instances Prices. <https://aws.amazon.com/ec2/spot/pricing/>, 2018.
- [3] Azure Batch. <https://docs.microsoft.com/en-us/azure/batch/batch-low-pri-vms>, 2018.
- [4] Azure Batch Pricing. <https://azure.microsoft.com/en-us/pricing/details/batch/>, 2018.
- [5] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard, M. Kudlur, J. Levenberg, R. Monga, S. Moore, D. G. Murray, B. Steiner, P. Tucker, V. Vasudevan, P. Warden, M. Wicke, Y. Yu, and X. Zheng. Tensorflow: A system for large-scale machine learning. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, pages 265–283, 2016.
- [6] R. C. Agarwal, S. M. Balle, F. G. Gustavson, M. Joshi, and P. Palkar. A three-dimensional approach to parallel matrix multiplication. *IBM Journal of Research and Development*, 39(5):575–582, 1995.
- [7] M. Aktas, P. Peng, and E. Soljanin. Effective straggler mitigation: which clones should attack and when? *ACM SIGMETRICS Performance Evaluation Review*, 45(2):12–14, 2017.
- [8] M. Aktas, P. Peng, and E. Soljanin. Straggler mitigation by delayed relaunch of tasks. *ACM SIGMETRICS Performance Evaluation Review*, 45(2):224–231, 2018.
- [9] M. Aliasgari, J. Kliever, and O. Simeone. Coded computation against straggling decoders for network function virtualization. *arXiv preprint arXiv:1709.01031*, 2017.
- [10] G. Ananthanarayanan, A. Ghodsi, S. Shenker, and I. Stoica. Effective straggler mitigation: Attack of the clones. In *NSDI*, volume 13, pages 185–198, 2013.
- [11] T. Baharav, K. Lee, O. Ocal, and K. Ramchandran. Straggler-proofing massive-scale distributed matrix multiplication with d-dimensional product codes.
- [12] M. Barnett, L. Shuler, R. van De Geijn, S. Gupta, D. G. Payne, and J. Watts. Interprocessor collective communication library (intercom). In *Proceedings of IEEE Scalable High Performance Computing Conference*, pages 357–364. IEEE, 1994.
- [13] R. Bitar and S. El Rouayheb. Staircase codes for secret sharing with optimal communication and read overheads. *IEEE Transactions on Information Theory*, 64(2):933–943, 2018.
- [14] R. Bitar, P. Parag, and S. El Rouayheb. Minimizing latency for secure distributed computing. In *IEEE International Symposium on Information Theory (ISIT)*, pages 2900–2904, 2017.
- [15] Z. Charles and D. Papailiopoulos. Gradient coding using the stochastic block model. In *2018 IEEE International Symposium on Information Theory (ISIT)*, pages 1998–2002. IEEE, 2018.
- [16] L. Chen, Z. Charles, D. Papailiopoulos, et al. DRACO: Robust distributed training via redundant gradients. *ICML*, 2018.
- [17] B.-G. Chun, T. Condie, Y. Chen, B. Cho, A. Chung, C. Curino, C. Douglas, M. Interlandi, B. Jeon, J. S. Jeong, G. Lee, Y. Lee, T. Majestro, D. Malkhi, S. Matushevych, B. Myers, M. Mykhailova, S. Narayanamurthy, J. Noor, R. Ramakrishnan, S. Rao, R. Sears, B. Sezgin, T. Um, J. Wang, M. Weimer, and Y. Yang. Apache REEF: Retainable evaluator execution framework. *ACM Trans. Comput. Syst.*, 35(2):5:1–5:31, Oct. 2017.
- [18] J. Dean and S. Ghemawat. MapReduce: simplified data processing on large clusters. *Communications of the ACM*, 51(1):107–113, 2008.
- [19] S. Dutta, Z. Bai, H. Jeong, T. M. Low, and P. Grover. A unified coded deep neural network training strategy based on generalized PolyDot codes. In *IEEE International Symposium on Information Theory (ISIT)*, pages 1585–1589, 2018.
- [20] S. Dutta, Z. Bai, T. M. Low, and P. Grover. Codenet: Training large neural networks in presence of soft-errors. 2018.
- [21] S. Dutta, V. Cadambe, and P. Grover. Short-Dot: Computing large linear transforms distributedly using coded short dot products. In *Advances In Neural Information Processing Systems (NIPS)*, pages 2092–2100, 2016.

- [22] S. Dutta, V. Cadambe, and P. Grover. Coded convolution for parallel and distributed computing within a deadline. In *IEEE International Symposium on Information Theory (ISIT)*, pages 2403–2407, 2017.
- [23] S. Dutta, M. Fahim, F. Haddadpour, H. Jeong, V. Cadambe, and P. Grover. On the optimal recovery threshold of coded matrix multiplication. *arXiv preprint arXiv:1801.10292*, 2018.
- [24] M. Fahim, H. Jeong, F. Haddadpour, S. Dutta, V. Cadambe, and P. Grover. On the optimal recovery threshold of coded matrix multiplication. In *55th Annual Allerton Conference on Communication, Control, and Computing (Allerton)*, pages 1264–1270, 2017.
- [25] N. Ferdinand and S. C. Draper. Hierarchical coded computation. In *2018 IEEE International Symposium on Information Theory (ISIT)*, pages 1620–1624.
- [26] N. S. Ferdinand and S. C. Draper. Anytime coding for distributed computation. In *54th Annual Allerton Conference on Communication, Control, and Computing (Allerton)*, pages 954–960, 2016.
- [27] K. Gardner, S. Zbarsky, S. Doroudi, M. Harchol-Balter, and E. Hyttia. Reducing latency via redundant requests: Exact analysis. *ACM SIGMETRICS Performance Evaluation Review*, 43(1):347–360, 2015.
- [28] F. Haddadpour, Y. Yang, V. Cadambe, and P. Grover. Cross-iteration coded computing for iterative algorithms. In *56th Annual Conference on Communication, Control, and Computing (Allerton)*, Oct 2018.
- [29] F. Haddadpour, Y. Yang, M. Chaudhari, V. R. Cadambe, and P. Grover. Straggler-resilient and communication-efficient distributed iterative linear solver. *arXiv preprint arXiv:1806.06140*, 2018.
- [30] C. N. Hadjicostis and G. C. Verghese. Coding approaches to fault tolerance in linear dynamic systems. *IEEE Trans. on Information Theory*, 51(1):210–228, 2005.
- [31] W. Halbawi, N. Azizan-Ruhi, F. Salehi, and B. Hassibi. Improving distributed gradient descent using Reed-Solomon codes. *IEEE International Symposium on Information Theory (ISIT)*, 2018.
- [32] B. Hindman, A. Konwinski, M. Zaharia, A. Ghodsi, A. D. Joseph, R. H. Katz, S. Shenker, and I. Stoica. Mesos: A platform for fine-grained resource sharing in the data center. In *NSDI*, volume 11, pages 22–22, 2011.
- [33] K. H. Huang and J. A. Abraham. Algorithm-based fault tolerance for matrix operations. *IEEE Trans. on Computers*, 100(6):518–528, 1984.
- [34] H. Jeong, T. M. Low, and P. Grover. Masterless coded computing: A fully-distributed coded FFT algorithm. *Communications, Control and Computing (Allerton)*, 2018.
- [35] H. Jeong, F. Ye, and P. grover. Locally recoverable coded matrix multiplication. *Communications, Control and Computing (Allerton)*, 2018.
- [36] C. Karakus, Y. Sun, and S. Diggavi. Encoded distributed optimization. In *IEEE International Symposium on Information Theory (ISIT)*, pages 2890–2894, 2017.
- [37] C. Karakus, Y. Sun, S. Diggavi, and W. Yin. Straggler mitigation in distributed optimization through data encoding. In *Advances in Neural Information Processing Systems (NIPS)*, pages 5440–5448, 2017.
- [38] J. Kosaian, K. Rashmi, and S. Venkataraman. Learning a code: Machine learning for approximate non-linear coded computation. *arXiv preprint arXiv:1806.01259*, 2018.
- [39] K. Lee, M. Lam, R. Pedarsani, D. Papailiopoulos, and K. Ramchandran. Speeding up distributed machine learning using codes. *IEEE Transactions on Information Theory*, 64(3):1514–1529, 2018.
- [40] K. Lee, R. Pedarsani, D. Papailiopoulos, and K. Ramchandran. Coded computation for multicore setups. In *IEEE International Symposium on Information Theory (ISIT)*, pages 2413–2417, 2017.
- [41] K. Lee, C. Suh, and K. Ramchandran. High-dimensional coded matrix multiplication. In *IEEE International Symposium on Information Theory (ISIT)*, pages 2418–2422, 2017.
- [42] L. Li. On the arithmetic operational complexity for solving Vandermonde linear equations. *Japan journal of industrial and applied mathematics*, 17(1):15–18, 2000.

- [43] S. Li, S. M. M. Kalan, Q. Yu, M. Soltanolkotabi, and A. S. Avestimehr. Polynomially coded regression: Optimal straggler mitigation via data encoding. *arXiv preprint arXiv:1805.09934*, 2018.
- [44] S. Li, M. Maddah-Ali, Q. Yu, and A. S. Avestimehr. A fundamental tradeoff between computation and communication in distributed computing. *IEEE Transactions on Information Theory*, 64(1):109–128, 2018.
- [45] K. Mahajan, M. Chowdhury, A. Akella, and S. Chawla. Dynamic query re-planning using QOOP. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 253–267, Carlsbad, CA, 2018. USENIX Association.
- [46] R. K. Maity, A. S. Rawat, and A. Mazumdar. Robust gradient descent via moment encoding with LDPC codes. *SysML Conference*, 2018.
- [47] A. Mallick, M. Chaudhari, and G. Joshi. Rateless codes for near-perfect load balancing in distributed matrix-vector multiplication. *arXiv preprint arXiv:1804.10331*, 2018.
- [48] X. Meng, J. Bradley, B. Yavuz, E. Sparks, S. Venkataraman, D. Liu, J. Freeman, D. Tsai, M. Amde, S. Owen, D. Xin, R. Xin, M. J. Franklin, R. Zadeh, M. Zaharia, and A. Talwalkar. Mllib: Machine learning in Apache Spark. *Journal of Machine Learning Research*, 17(34):1–7, 2016.
- [49] S. Narayanamurthy, M. Weimer, D. Mahajan, T. Condie, S. Sellamanickam, and S. S. Keerthi. Towards resource-elastic machine learning. In *NIPS 2013 BigLearn Workshop*, 2013.
- [50] E. Ozfaturay, D. Gunduz, and S. Ulukus. Speeding up distributed gradient descent by utilizing non-persistent stragglers. *arXiv preprint arXiv:1808.02240*, 2018.
- [51] H. Park, K. Lee, J.-y. Sohn, C. Suh, and J. Moon. Hierarchical coding for distributed computing. *arXiv preprint arXiv:1801.04686*, 2018.
- [52] N. Pippenger. On networks of noisy gates. In *IEEE Annual Symposium on Foundations of Computer Science*, pages 30–38, 1985.
- [53] S. Prakash, A. Reisizadeh, R. Pedarsani, and S. Avestimehr. Coded computing for distributed graph analytics. *ISIT*, 2018.
- [54] N. Raviv, I. Tamo, R. Tandon, and A. G. Dimakis. Gradient coding from cyclic MDS codes and expander graphs. *arXiv preprint arXiv:1707.03858*, 2017.
- [55] A. Reisizadeh, S. Prakash, R. Pedarsani, and A. S. Avestimehr. Coded computation over heterogeneous clusters. In *IEEE International Symposium on Information Theory (ISIT)*, pages 2408–2412, 2017.
- [56] A. Sergeev and M. D. Balso. Horovod: fast and easy distributed deep learning in tensorflow. *CoRR*, abs/1802.05799, 2018.
- [57] A. Severinson, A. G. i Amat, and E. Rosnes. Block-diagonal and LT codes for distributed computing with straggling servers. *IEEE Transactions on Communications*, 2018.
- [58] U. Sheth, S. Dutta, M. Chaudhari, H. Jeong, Y. Yang, J. Kohonen, T. Roos, and P. Grover. An application of storage-optimal MatDot codes for coded matrix multiplication: Fast k-nearest neighbors estimation. *arXiv preprint arXiv:1811.11811*, 2018.
- [59] E. Solomonik and J. Demmel. Communication-optimal parallel 2.5 D matrix multiplication and LU factorization algorithms. In *European Conference on Parallel Processing*, pages 90–109. Springer, 2011.
- [60] G. Suh, K. Lee, and C. Suh. Matrix sparsification for coded matrix multiplication. In *Communication, Control, and Computing (Allerton)*, pages 1271–1278, 2017.
- [61] P. L. Suresh, M. Canini, S. Schmid, and A. Feldmann. C3: Cutting tail latency in cloud data stores via adaptive replica selection. In *12th USENIX Symposium on Networked Systems Design and Implementation*, pages 513–527. USENIX Association, 2015.
- [62] R. Tandon, Q. Lei, A. G. Dimakis, and N. Karampatziakis. Gradient coding: Avoiding stragglers in distributed learning. In *International Conference on Machine Learning (ICML)*, pages 3368–3376, 2017.
- [63] L. Tang, K. Konstantinidis, and A. Ramamoorthy. Erasure coding for distributed matrix multiplication for matrices with bounded entries. *IEEE Communications Letters*, 2018.

- [64] M. G. Taylor. Reliable Information Storage in Memories Designed from Unreliable Components. *Bell Syst. Tech. J.*, 47(10):2299–2337, 1968.
- [65] V. K. Vavilapalli, A. C. Murthy, C. Douglas, S. Agarwal, M. Konar, R. Evans, T. Graves, J. Lowe, H. Shah, and S. Seth. Apache hadoop yarn: Yet another resource negotiator. In *Proceedings of the 4th annual Symposium on Cloud Computing*, page 5, 2013.
- [66] J. Von Neumann. Probabilistic logics and the synthesis of reliable organisms from unreliable components. *Automata Studies*, 34:43–98, 1956.
- [67] D. Wang, G. Joshi, and G. Wornell. Efficient task replication for fast response times in parallel computation. In *ACM SIGMETRICS Performance Evaluation Review*, volume 42, pages 599–600, 2014.
- [68] D. Wang, G. Joshi, and G. Wornell. Using straggler replication to reduce latency in large-scale parallel computing. In *ACM SIGMETRICS Performance Evaluation Review*, volume 43, pages 7–11, 2015.
- [69] S. Wang, J. Liu, and N. Shroff. Coded sparse matrix multiplication. *ICML*, 2018.
- [70] H. Yang and J. Lee. Secure distributed computing with straggling servers using polynomial codes. *IEEE Transactions on Information Forensics and Security*, 14(1):141–150, 2019.
- [71] Y. Yang, M. Chaudhari, P. Grover, and S. Kar. Coded iterative computing using substitute decoding. *arXiv preprint arXiv:1805.06046*, 2018.
- [72] Y. Yang, P. Grover, and S. Kar. Coded distributed computing for inverse problems. In *Advances in Neural Information Processing Systems (NIPS)*, pages 709–719, 2017.
- [73] Y. Yang, P. Grover, and S. Kar. Computing linear transformations with unreliable components. *IEEE Transactions on Information Theory*, 63(6), 2017.
- [74] M. Ye and E. Abbe. Communication-computation efficient gradient coding. *ICML*, 2018.
- [75] Q. Yu, M. Maddah-Ali, and S. Avestimehr. Polynomial codes: an optimal design for high-dimensional coded matrix multiplication. In *Advances in Neural Information Processing Systems*, pages 4403–4413, 2017.
- [76] Q. Yu, M. A. Maddah-Ali, and A. S. Avestimehr. Straggler mitigation in distributed matrix multiplication: Fundamental limits and optimal coding. *arXiv preprint:1801.07487*, 2018.
- [77] Q. Yu, N. Raviv, J. So, and A. S. Avestimehr. Lagrange coded computing: Optimal design for resiliency, security and privacy. *Workshop on Systems for ML and Open Source Software at NeurIPS*, 2018.